



INFINEON / BIT EXCELLENCE PROGRAM — 2026

WATER SUPPLY APP

Weekly Progress Report

A plain-language account of the backend work completed this week — written so that both engineers and non-technical stakeholders can follow what changed and why it matters.

Week of September 7–11, 2026

Backend Team — Progress Report

1. Summary

This week, the backend team focused on getting the foundations of the application right, rather than adding new visible features. In short: we made sure the application actually starts up correctly, we fixed several everyday rules that keep the data trustworthy, we began reorganizing the code so it stays manageable as the project grows, and we prepared the ground for two things that will matter soon — connecting with the other modules the rest of the team is building, and adding proper security once that connection work is stable. We also strengthened the automated tests that catch mistakes early, and made the database easier to evolve safely over time.

None of this is visible to an end user yet, in the way a new screen or button would be. But it is the kind of work that determines whether everything built on top of the backend later on will be reliable, or whether it will need to be redone.

2. Introduction

Until this week, the backend had a blocking problem: the application and its database were not configured to speak the same language, which meant the whole system could fail to start in certain environments. Beyond that immediate blocker, the code that handles core operations (farms, water sources, irrigation, quality checks) had grown organically, with shortcuts that made sense early on but now make the system harder to extend safely and harder for other developers to plug into.

This report explains, in plain terms, what was done about each of these issues this week. Where a technical term is unavoidable, we explain what it means the first time it appears.

3. What We Worked On This Week

The work is organized into seven areas. Each one is explained below: what the problem or goal was, why it matters, and what was actually done.

3.1 Getting the Application to Start Reliably

This was the top priority, because it was a blocker: without it, nothing else can be tested or demonstrated. The problem was simple to describe but important to fix: the application's configuration file expected the database connection details under one set of names (for example, "DB_URL"), while the file that starts the database container provided them under different names (for example, "DB_HOST" and "POSTGRES_DB"). It is a bit like two people trying to hand each other a package using two different addresses for the same building — the delivery fails even though both sides did their job correctly.

We aligned the two sides so they use matching names, and verified this by starting the whole application stack from scratch (database included) and checking that the backend actually connects and stays up, rather than assuming it would work.

3.2 Making the Core Business Rules Behave Correctly

"Business rules" simply means the everyday logic that keeps information correct and consistent — for example, what should happen if someone tries to delete a water source that is still being used elsewhere

in the system, or what the application should reply if someone asks for something that doesn't exist. Several such situations were not yet handled consistently.

This week's work included:

- Making sure a meaningful, correctly formatted error is returned when someone asks for an entity (a farm, a field, a water source, etc.) that doesn't exist, instead of a confusing or generic failure.
- Preventing the deletion of entities that are still referenced elsewhere, so the data can't end up pointing to something that no longer exists.
- Checking that the relationships between farms, fields and zones make sense before accepting them.
- Standardizing how the application reports different kinds of errors (invalid input, not found, conflicting data, unexpected failure), so every part of the system communicates problems the same way.
- Improving how the application reacts when the external weather service doesn't respond properly, instead of letting that failure spread.
- Making sure report dates are calculated fresh each time they are requested, rather than reused from an earlier moment.
- Replacing generic, loosely structured responses with clearly defined ones, so the shape of the data returned by the API is predictable and documented.
- Handling dates, measurement units and time zones consistently, so a value entered in one place is interpreted the same way everywhere else.

Taken together, these fixes reduce the number of situations where the application could silently produce incorrect or inconsistent results.

3.3 Reorganizing the Code into Clear Layers

Right now, parts of the code allow a request coming into the application to reach directly into the database, skipping any place where business logic or checks could happen. This works in the short term but becomes risky as more features are added, because the same checks end up duplicated — or missed — in different places.

The fix is to organize the code into a clear chain of responsibility, a pattern commonly described as Controller → Service → Repository → Entity: the Controller only receives the request, the Service applies the business rules and decisions, the Repository is the only part allowed to talk to the database, and the Entity represents the actual piece of data (a farm, a water source, etc.). Think of it like a restaurant: the waiter (Controller) takes the order, the kitchen (Service) decides how to prepare it correctly, and the storeroom (Repository) is the only place ingredients are actually taken from — the waiter never goes rummaging through the storeroom directly.

This week, this reorganization began with the areas that everything else depends on — farms, fields and zones — before moving on to water sources, consumption and quality, then irrigation, then notifications and reports, in that order.

3.4 Preparing the Backend to Connect with Other Modules

Full user login and permissions are intentionally not being built yet (see 3.5), but the backend still needs to be shaped so that connecting it to the modules other teams are building — and adding proper security later — doesn't require tearing everything apart and starting over.

Concretely, this means: grouping all the application's web addresses ("routes") under one stable, versioned pattern; never exposing the raw internal data structures directly to the outside world, and instead defining separate, clearly documented request and response formats; using one consistent, predictable format whenever something goes wrong; documenting exactly what each part of the API expects and returns, including date formats and units; and keeping the water-supply-specific logic cleanly separated so future modules can attach to the system without becoming entangled with it.

3.5 Security — Deliberately On Hold, and Why

Adding real authentication and permissions (deciding who is allowed to log in, and what each type of user is allowed to do or change) has not started yet, and this is an intentional decision rather than something overlooked.

The reason is that security choices — which login system to use, how sessions and tokens work, which roles exist, and which actions each role can perform — depend directly on the business logic being built by the other modules across the project. Locking in a security model before that logic has matured elsewhere would risk designing permissions around rules that are still going to change, which would mean redoing the security layer more than once. The plan is to wait until the other modules have progressed enough with their own business logic that a shared, stable security model can be defined once, correctly, for the whole system — rather than module by module.

This item remains on the roadmap and has not been forgotten: choosing an identity provider and token format, defining shared roles and permissions, finishing login/refresh/logout/reset flows, protecting every action that changes data, and testing both authorized and unauthorized access will all follow once that shared decision point is reached.

3.6 Making the Database More Solid

A database migration is simply a small, recorded, controlled change to the structure of the database (adding a column, a table, a constraint) — recorded so that anyone, at any time, can rebuild the database from scratch and get the exact same structure. This week's work moves the project toward always changing the database this way, using a tool called Flyway, instead of editing the original setup script after it has already been used — which would make different environments quietly drift apart from each other.

Alongside this, we are adding the missing links and lookup indexes between tables (so related data stays connected and searches stay fast), adding the ability to fetch only a page of results and to filter them at the database level, and gradually replacing the pattern of "load everything, then filter it in the application" with targeted database queries — which matters more and more as the amount of data grows. We are also considering a "soft delete" for certain records, meaning they are marked as removed rather than physically erased, which matters for things still referenced elsewhere. A small, controlled set of demo data is being prepared as well, to make the system easy to try out consistently.

3.7 Strengthening Testing

Automated tests are small programs that check the application behaves correctly, without a person needing to click through it by hand every time. The current coverage is not sufficient, so this week's effort increases it across several levels: individual pieces of business logic tested in isolation; the web layer tested on its own; full integration tests run against a real database (ideally using disposable, temporary database instances created just for testing, known as Testcontainers); security-related tests once that layer exists; complete create/read/update/delete checks; the quality-monitoring and alerting rules; the reporting logic; both valid and invalid responses from the external weather service; and a test that the whole application actually starts up cleanly from nothing.

The goal is that a change made anywhere in the system is caught automatically if it accidentally breaks something else, rather than being discovered later by a user.

4. At a Glance

The table below summarizes where each area stands, without the technical detail — useful for anyone who wants the short version.

Area	What it means, in plain terms	Status this week
Startup & environment	Making sure the application and its database actually agree on how to talk to each other	Fixed — top priority, resolved
Core business rules	The everyday logic that keeps data correct (e.g. can't delete something still in use)	Fixed
Code organization	Splitting responsibilities so the code doesn't turn into a tangled mess	Fixed, farms/fields/zones first
Multi-module readiness	Preparing stable "plugins" so other teams' modules can connect later	Fixed
Security	Login, permissions and access protection	Deliberately on hold
Database reliability	Making changes to the database traceable, fast and safe	Fixed
Testing	Automated checks that catch problems before users do	In progress, coverage growing

"In progress" means active, ongoing work this week — not delayed or blocked.

5. How We Checked That Everything Works

Rather than assuming the fixes worked, each change was verified directly:

- The full application stack (backend and database together) was rebuilt and started from a clean state, to confirm the startup issue described in 3.1 is genuinely resolved and not just worked around.
- The status of every running component was checked, along with the backend's own logs, to catch silent failures.

- The automated test suite was run against a real, active database, rather than a simulated one, so the results reflect how the application will actually behave.

This matters because a fix that only works "on one machine" or only against a simplified test setup isn't reliable enough to build on.

6. What's Still Fragile, and What Comes Next

A few points are worth flagging honestly rather than glossing over:

- Security is not yet in place at all — this is expected at this stage (see 3.5), but it means the application is not ready for any real user data until that work happens.
- The reorganization into clear layers (3.3) is still in progress for several areas; until it's complete, some parts of the code remain more fragile and harder to extend than others.
- Test coverage, while improving, is not yet complete across every feature, so some issues could still slip through undetected for now.

The next steps follow naturally from this week's work: continue the layer-by-layer reorganization through water sources, irrigation, and notifications; keep expanding automated test coverage; continue hardening the database with further migrations; and revisit the security roadmap as soon as the other modules' business logic is mature enough to design it properly, once, for the whole system.

7. Conclusion

This week's work was foundational rather than visible: getting the application to start reliably, tightening the everyday rules that keep data correct, beginning a cleaner internal structure, and preparing — without yet building — the connections to other modules and to security. None of this changes what an end user sees today, but it directly determines how smoothly the next stages of the project — including the features other teams are relying on the backend for — will go.